

Laboratory Work 1: Relational Model & Keys

Database Systems · Week 2 · Complete solutions
Student: Zheksembaev Erbol

Part	Tasks	Contents
Part 1	1.1, 1.2	Superkeys, candidate keys, primary keys, foreign keys
Part 2	2.1, 2.2	ER diagrams: hospital system, e-commerce platform
Part 4	4.1, 4.2	Normalization: 1NF → 2NF → 3NF, BCNF decomposition
Part 5	5.1	Design challenge: student clubs (ER diagram, schema, design decisions, queries)

Conventions. Primary-key attributes are underlined; foreign keys are marked *FK* → *Table*. Data types follow standard SQL (PostgreSQL style). Assumptions are stated wherever the requirements leave a choice.

Part 1. Key Identification Exercises

Task 1.1 Superkey and Candidate Key Analysis

Definitions. A **superkey** is any set of attributes whose values uniquely identify every tuple of a relation. A **candidate key** is a *minimal* superkey: if any attribute is removed, it is no longer unique. The **primary key** is the one candidate key chosen by the designer. Keys come from the meaning of the data (business rules); sample rows can only disprove uniqueness, never prove it.

Relation A: Employee(EMPID, SSN, Email, Phone, Name, Department, Salary)

Uniqueness of each attribute. EMPID is assigned by the company to exactly one employee. SSN identifies one person nationally. Email is a personal company mailbox, so it is unique per employee. Name, Department and Salary can repeat (the sample already shows Department = IT twice). Phone is discussed in question 4 and is not treated as unique.

1. Superkeys (at least 6). Every set that contains EMPID, SSN or Email is a superkey:

#	Superkey	Why it is unique
1	{EMPID}	EMPID alone is unique
2	{SSN}	SSN alone is unique
3	{Email}	Email alone is unique
4	{EMPID, Name}	contains EMPID
5	{SSN, Department}	contains SSN
6	{Email, Salary}	contains Email
7	{EMPID, SSN, Email}	contains three keys (unique but not minimal)
8	{EMPID, SSN, Email, Phone, Name, Department, Salary}	the set of all attributes is always a superkey

In total there are $2^7 - 2^4 = 128 - 16 = \mathbf{112}$ **superkeys**: all 128 subsets of the 7 attributes, minus the 16 subsets built only from {Phone, Name, Department, Salary}, which contain no unique attribute.

2. Candidate keys: {EMPID}, {SSN}, {Email}. Each consists of one attribute, so it is automatically minimal. A set such as {EMPID, Name} is a superkey but *not* a candidate key, because Name can be removed and the set stays unique.

3. Primary key: EmpID. It is short and numeric (fast indexes and joins), it is controlled by the company and never changes. SSN is sensitive personal data: using it as a key would copy it into every referencing table, and a foreign employee may not have one. Email can change (marriage, name change, domain change), which would force updates in all foreign keys.

4. Can two employees have the same phone number? Yes, this is possible. In the sample the three phone numbers (555-0101, 555-0102, 555-0103) are different, but three rows cannot prove a rule for all future data. In reality employees may share an office or department line, and a number can be reassigned to a new employee. Therefore Phone is **not** a candidate key, and the database should not declare it UNIQUE.

Relation B: Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum primary key: {StudentID, CourseCode, Section, Semester, Year}

Using the rules: “a student cannot register for the same course section in the same semester” means that the combination student + course + section + semester (of a particular year) occurs at most once, so this set is unique.

2. Why each attribute is necessary (removing it would allow two legal rows with the same key):

Attribute	Why it is needed	Example of a clash without it
StudentID	Many students register for the same section.	Ali and Dana both in CS101-01, Fall 2026.
CourseCode	Section numbers such as 01 repeat in every course, and a student takes several courses per term.	Ali in CS101-01 and MATH201-01, Fall 2026.
Section	The uniqueness rule is stated at section level and credits are fixed per section, so a registration is for a specific section. Nothing forbids registering in two different sections of one course.	Ali in CS101-01 and CS101-02 (e.g. lecture and lab), Fall 2026.
Semester	A student can take the same course in different semesters (retake).	Ali in CS101-01 in Fall 2026 and Spring 2026.
Year	A semester name (Fall, Spring) repeats every year.	Ali in CS101-01 in Fall 2025 and again in Fall 2026.

Assumptions. (a) If Semester already contained the year (e.g. “Fall-2026”), Year would be unnecessary. (b) If the university forbade taking two sections of the same course in one semester, Section could be removed from the key.

3. Additional candidate keys: none. Grade and Credits are descriptive values that repeat (many students get an A; many sections have 5 credits), so no other attribute combination can replace the key. Every other superkey is the key plus extra attributes. **Observation:** the rule “each course section in a semester has a fixed credit value” gives the functional dependency {CourseCode, Section, Semester, Year} → Credits. It depends on only part of the key, which is a 2NF violation; Credits should be moved to a separate CourseSection table.

Task 1.2 Foreign Key Design

Primary keys of the given tables: Student(StudentID), Professor(ProfID), Course(CourseID), Department(DeptCode), Enrollment(StudentID, CourseID, Semester).

#	Foreign key	References	Meaning	Cardinality
1	Student.AdvisorID	Professor.ProfID	Each student's advisor is a professor.	N students : 1 professor
2	Professor.Department	Department.DeptCode	The department a professor works in (the column stores the department code).	N professors : 1 department

#	Foreign key	References	Meaning	Cardinality
3	Course.DepartmentCode	Department.DeptCode	The department that offers the course.	N courses : 1 department
4	Department.ChairID	Professor.ProfID	The chair of a department is a professor.	1 : 1 (ChairID should be UNIQUE)
5	Enrollment.StudentID	Student.StudentID	Who is enrolled.	N enrollments : 1 student
6	Enrollment.CourseID	Course.CourseID	Which course is taken.	N enrollments : 1 course

Notes.

- A foreign key does not need the same name as the key it references (DepartmentCode → DeptCode, AdvisorID → ProfID); it needs the same domain and data type.
- In Enrollment the foreign keys StudentID and CourseID are also part of the primary key: Enrollment implements the M:N relationship between Student and Course.
- **Circular reference:** Professor → Department (FK 2) and Department → Professor (FK 4). To insert the first rows, create both tables, allow Department.ChairID to be NULL (or add this FK later with ALTER TABLE), insert departments, then professors, then set the chairs.
- **Design option:** Student.Major could also reference Department.DeptCode if majors are recorded by department code. As given, Major is a descriptive text value, so it is not required to be a foreign key.

Part 2. ER Diagram Construction

All diagrams use **Chen notation**. Legend:

Symbol	Meaning	Symbol	Meaning
Rectangle	Strong entity	Double rectangle	Weak entity
Diamond	Relationship	Double diamond	Identifying relationship (weak entity ↔ owner)
Ellipse	Simple attribute	Ellipse with sub-ellipses	Composite attribute
Double ellipse	Multi-valued attribute	Dashed ellipse	Derived attribute
<u>Underlined</u>	Primary key	<i><u>Italic underlined</u></i>	Partial key of a weak entity (dashed underline in classic Chen notation)
Double line	Total participation (every entity must take part)	Single line	Partial participation (optional)
1 / N / M	Cardinality written next to the entity		

Task 2.1 Hospital Management System

1. Entities (strong and weak)

Entity	Type	Justification
PATIENT	Strong	Identified by its own unique PatientID.
DOCTOR	Strong	Identified by its own unique DoctorID.
DEPARTMENT	Strong	Identified by its own unique DeptCode.
APPOINTMENT	Strong	Has its own AppointmentID. Modelled as an entity, not as a plain M:N relationship, because the same patient can see the same doctor many times; a plain relationship allows only one instance per patient-doctor pair.
PRESCRIPTION	Strong	Has its own PrescriptionID; links a doctor, a patient and a medication.
MEDICATION	Strong	Has its own MedicationID. A separate entity so each medication name is stored once instead of being retyped in every prescription.
ROOM	Weak	A room number is unique only inside a department (room 101 exists in Cardiology and in Neurology). A room is identified by its owner's key plus its partial key: DeptCode + RoomNumber. Owner: DEPARTMENT; identifying relationship: HAS.

2. Attributes and their classification

Entity	Attribute	Classification
PATIENT	<u>PatientID</u>	Simple, single-valued, primary key
	Name	Simple (could be composite: FirstName, LastName)
	BirthDate	Simple, stored
	Age	Derived (computed from BirthDate and today's date; not stored)
	Address	Composite : Street, City, State, Zip
	Phone	Multi-valued (multiple numbers allowed)
	Insurance	Composite : Provider, PolicyNo
DOCTOR	<u>DoctorID</u>	Simple, primary key
	Name	Simple

Entity	Attribute	Classification
	Specialization	Multi-valued (a doctor can have several)
	Phone	Simple (assumption: one work phone; the requirements allow multiple phones only for patients)
	OfficeLocation	Simple
DEPARTMENT	<u>DeptCode</u>	Simple, primary key
	DeptName, Location	Simple
ROOM	<u>RoomNumber</u>	Simple, partial key (unique only within one department)
APPOINTMENT	<u>AppointmentID</u>	Simple, primary key
	DateTime, Purpose, Notes	Simple (Notes is optional)
PRESCRIPTION	<u>PrescriptionID</u>	Simple, primary key
	DateIssued, Dosage, Instructions	Simple
MEDICATION	<u>MedicationID</u>	Simple, primary key
	MedName	Simple

3. Relationships and cardinalities

Relationship	Entities	Cardinality	Participation	Meaning
HAS (identifying)	DEPARTMENT – ROOM	1 : N	ROOM total; DEPARTMENT partial	A department has many rooms; a room belongs to exactly one department.
WORKS_IN	DOCTOR – DEPARTMENT	N : 1	DOCTOR total; DEPARTMENT partial	Each doctor works in one department (assumption); a department has many doctors.
BOOKS	PATIENT – APPOINTMENT	1 : N	APPOINTMENT total	A patient can have many appointments; each appointment is for one patient.
CONDUCTS	DOCTOR – APPOINTMENT	1 : N	APPOINTMENT total	A doctor has many appointments; each appointment is with one doctor.
WRITES	DOCTOR – PRESCRIPTION	1 : N	PRESCRIPTION total	A doctor writes many prescriptions; each is written by one doctor.
RECEIVES	PATIENT – PRESCRIPTION	1 : N	PRESCRIPTION total	A patient receives many prescriptions; each is for one patient.
FOR	PRESCRIPTION – MEDICATION	N : 1	PRESCRIPTION total	Each prescription is for one medication; a medication appears in many prescriptions.

Through APPOINTMENT (and PRESCRIPTION) the relationship between PATIENT and DOCTOR is **M:N**: a patient sees many doctors, and a doctor sees many patients.

4. ER diagram

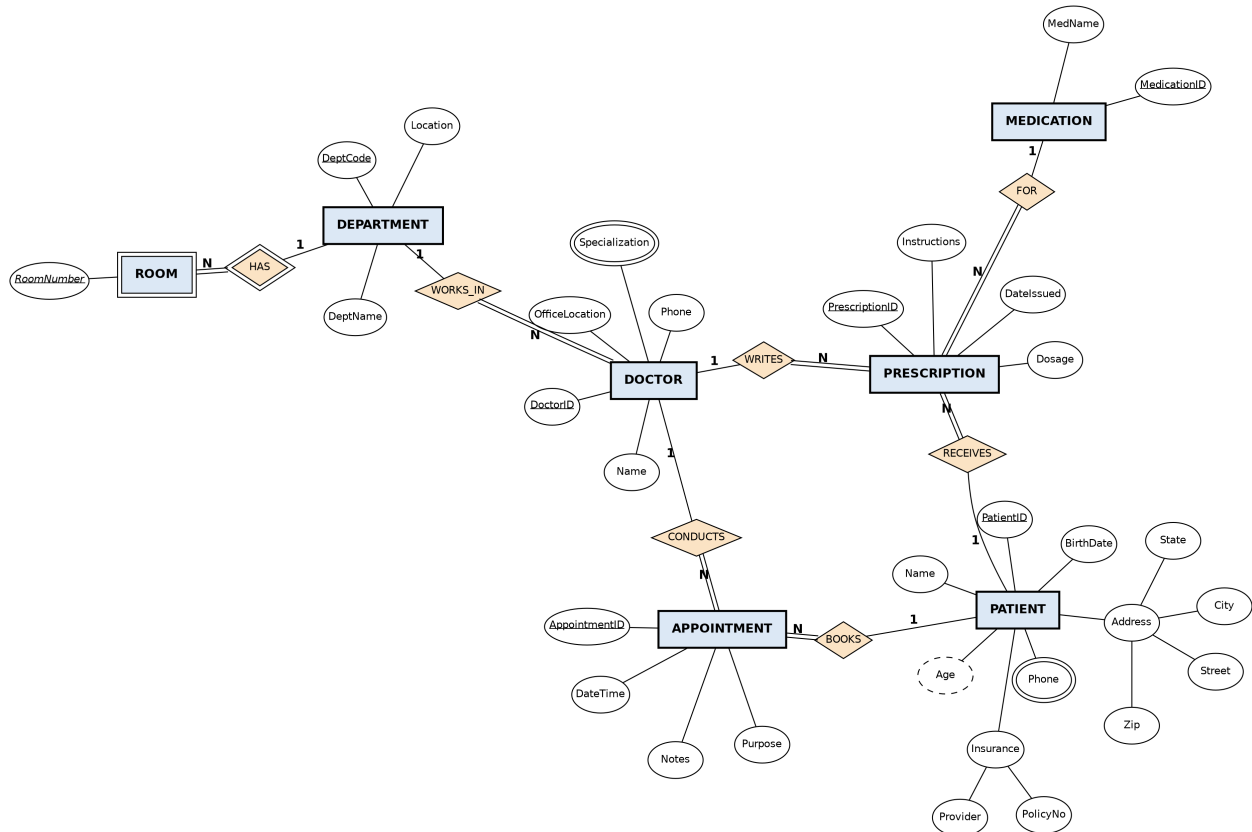


Figure 1. ER diagram of the hospital management system (Chen notation).

5. Primary keys

PATIENT: PatientID; DOCTOR: DoctorID; DEPARTMENT: DeptCode; APPOINTMENT: AppointmentID; PRESCRIPTION: PrescriptionID; MEDICATION: MedicationID; ROOM (weak): DeptCode + RoomNumber (owner key + partial key). Keys are underlined in Figure 1.

Relational mapping (for reference). Multi-valued attributes become separate tables; composite attributes are flattened; the derived attribute Age is not stored.

Table	Definition (PK underlined, FK marked)
Department	<u>DeptCode</u> VARCHAR(10), DeptName VARCHAR(100) NOT NULL, Location VARCHAR(100)
Doctor	<u>DoctorID</u> INT, Name VARCHAR(100) NOT NULL, Phone VARCHAR(20), OfficeLocation VARCHAR(50), DeptCode VARCHAR(10) NOT NULL FK → Department
DoctorSpecialization	<u>DoctorID</u> INT FK → Doctor, <u>Specialization</u> VARCHAR(60)
Patient	<u>PatientID</u> INT, Name VARCHAR(100) NOT NULL, BirthDate DATE NOT NULL, Street VARCHAR(100), City VARCHAR(50), State VARCHAR(50), Zip VARCHAR(10), InsuranceProvider VARCHAR(100), PolicyNo VARCHAR(40)
PatientPhone	<u>PatientID</u> INT FK → Patient, <u>Phone</u> VARCHAR(20)
Room	<u>DeptCode</u> VARCHAR(10) FK → Department, <u>RoomNumber</u> VARCHAR(10)
Appointment	<u>AppointmentID</u> INT, PatientID INT NOT NULL FK → Patient, DoctorID INT NOT NULL FK → Doctor, AppointmentDateTime TIMESTAMP NOT NULL, Purpose VARCHAR(200), Notes TEXT
Medication	<u>MedicationID</u> INT, MedName VARCHAR(100) NOT NULL UNIQUE

Table	Definition (PK underlined, FK marked)
Prescription	<u>PrescriptionID</u> INT, DoctorID INT NOT NULL <i>FK</i> → <i>Doctor</i> , PatientID INT NOT NULL <i>FK</i> → <i>Patient</i> , MedicationID INT NOT NULL <i>FK</i> → <i>Medication</i> , DateIssued DATE NOT NULL, Dosage VARCHAR(50) NOT NULL, Instructions TEXT

Task 2.2 E-commerce Platform

Assumptions: each product belongs to one category and is supplied by one vendor; a customer writes at most one review per product; inventory is tracked in one warehouse, so each product has one inventory record; each order is shipped to one address.

1. ER diagram

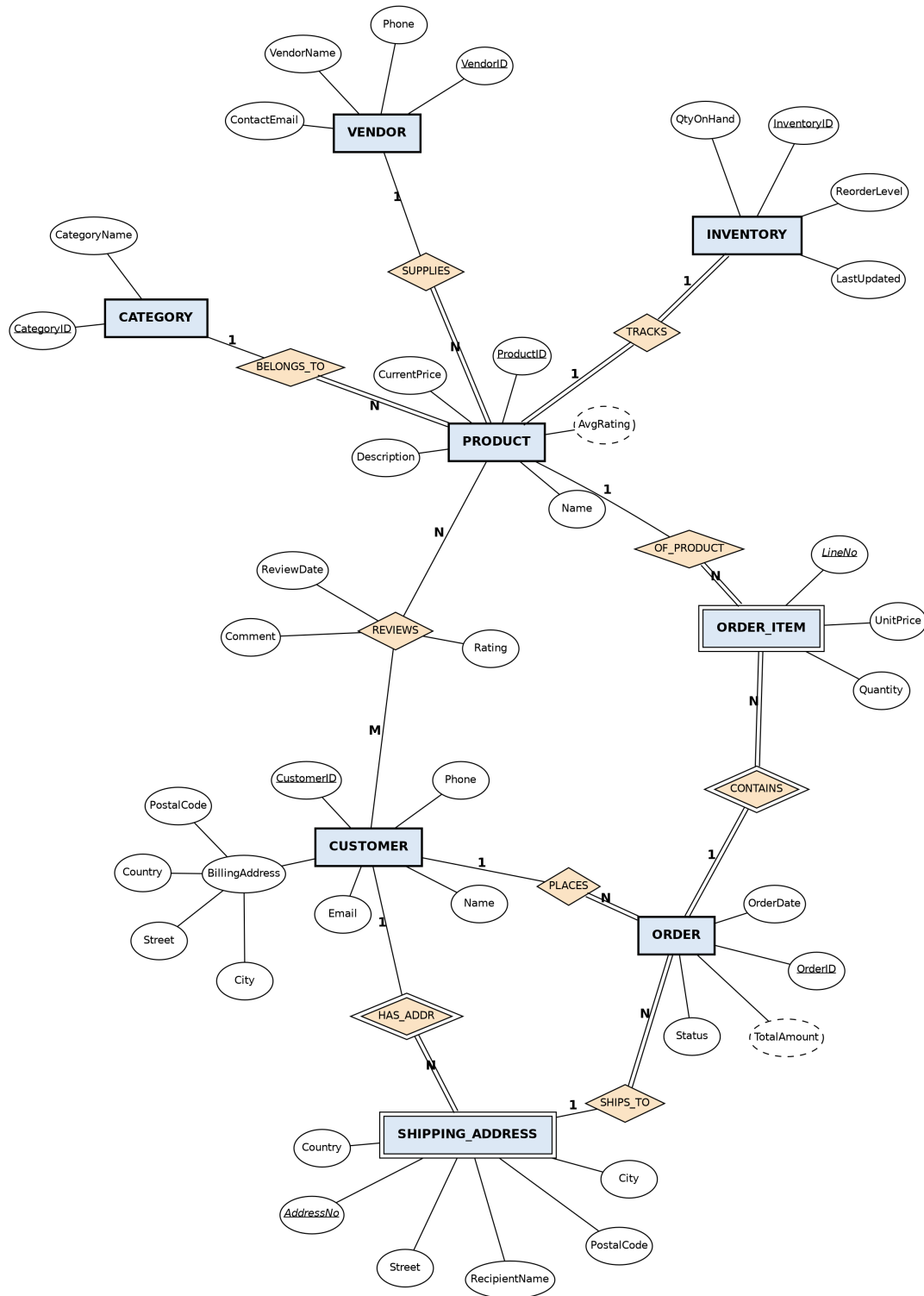


Figure 2. ER diagram of the e-commerce platform (Chen notation).

Relationship	Entities	Cardinality	Participation
PLACES	CUSTOMER - ORDER	1 : N	ORDER total (every order has a customer)
HAS_ADDR (identifying)	CUSTOMER - SHIPPING_ADDRESS	1 : N	SHIPPING_ADDRESS total
SHIPS_TO	ORDER - SHIPPING_ADDRESS	N : 1	ORDER total
CONTAINS (identifying)	ORDER - ORDER_ITEM	1 : N	both total (an order has at least one item)
OF_PRODUCT	ORDER_ITEM - PRODUCT	N : 1	ORDER_ITEM total
BELONGS_TO	PRODUCT - CATEGORY	N : 1	PRODUCT total
SUPPLIES	VENDOR - PRODUCT	1 : N	PRODUCT total
TRACKS	PRODUCT - INVENTORY	1 : 1	both total
REVIEWS	CUSTOMER - PRODUCT	M : N	partial (attributes: Rating, Comment, ReviewDate)

The billing address is a composite attribute of CUSTOMER, while shipping addresses are a separate weak entity. This satisfies the requirement that the shipping address can differ from the billing address, and a customer can keep several shipping addresses.

2. Weak entity and justification

ORDER_ITEM is a weak entity. Its partial key LineNo (1, 2, 3, ...) restarts in every order, so LineNo alone cannot identify an item: line 1 exists in every order. An order item also cannot exist without its order: deleting the order deletes its lines. Its full key is therefore the owner's key plus the partial key: (**OrderID**, **LineNo**). It is connected to its owner ORDER by the identifying relationship CONTAINS with total participation.

SHIPPING_ADDRESS is a second weak entity: address number 1, 2, ... is unique only per customer, and an address has no meaning without its customer. Key: (CustomerID, AddressNo).

3. Many-to-many relationship that needs attributes

REVIEWS (CUSTOMER M:N PRODUCT) with attributes **Rating**, **Comment**, **ReviewDate**. A customer reviews many products, and a product is reviewed by many customers. A rating belongs to the *pair* (customer, product): it cannot be stored in CUSTOMER (a customer gives different ratings to different products) or in PRODUCT (a product gets different ratings from different customers). In tables it becomes Review(CustomerID, ProductID, Rating, Comment, ReviewDate).

The link between ORDER and PRODUCT is also M:N with attributes **Quantity** and **UnitPrice**; it is implemented through ORDER_ITEM. UnitPrice is the price at the moment of ordering, which is different from the current Product.CurrentPrice; it is historical data, not redundancy.

Relational mapping (for reference). "Order" is a reserved word in SQL, so the table is named CustomerOrder. Derived attributes TotalAmount and AvgRating are computed by queries, not stored.

Table	Definition (PK underlined, FK marked)
Customer	<u>CustomerID</u> INT, Name VARCHAR(100) NOT NULL, Email VARCHAR(100) NOT NULL UNIQUE, Phone VARCHAR(20), BillStreet VARCHAR(100), BillCity VARCHAR(50), BillPostalCode VARCHAR(10), BillCountry VARCHAR(50)
ShippingAddress	<u>CustomerID</u> INT FK → Customer, <u>AddressNo</u> INT, RecipientName VARCHAR(100), Street VARCHAR(100) NOT NULL, City VARCHAR(50) NOT NULL, PostalCode VARCHAR(10), Country VARCHAR(50) NOT NULL
Category	<u>CategoryID</u> INT, CategoryName VARCHAR(60) NOT NULL UNIQUE
Vendor	<u>VendorID</u> INT, VendorName VARCHAR(100) NOT NULL, ContactEmail VARCHAR(100), Phone VARCHAR(20)

Table	Definition (PK underlined, FK marked)
Product	<u>ProductID</u> INT, Name VARCHAR(120) NOT NULL, Description TEXT, CurrentPrice DECIMAL(10,2) NOT NULL CHECK (CurrentPrice ≥ 0), CategoryID INT NOT NULL FK → <i>Category</i> , VendorID INT NOT NULL FK → <i>Vendor</i>
Inventory	<u>InventoryID</u> INT, ProductID INT NOT NULL UNIQUE FK → <i>Product</i> , QtyOnHand INT NOT NULL CHECK (QtyOnHand ≥ 0), ReorderLevel INT, LastUpdated TIMESTAMP
CustomerOrder	<u>OrderID</u> INT, CustomerID INT NOT NULL FK → <i>Customer</i> , AddressNo INT NOT NULL, OrderDate TIMESTAMP NOT NULL, Status VARCHAR(20); (CustomerID, AddressNo) FK → <i>ShippingAddress</i> — guarantees the address belongs to the same customer
OrderItem	<u>OrderID</u> INT FK → <i>CustomerOrder</i> , <u>LineNo</u> INT, ProductID INT NOT NULL FK → <i>Product</i> , Quantity INT NOT NULL CHECK (Quantity > 0), UnitPrice DECIMAL(10,2) NOT NULL
Review	<u>CustomerID</u> INT FK → <i>Customer</i> , <u>ProductID</u> INT FK → <i>Product</i> , Rating SMALLINT NOT NULL CHECK (Rating BETWEEN 1 AND 5), Comment TEXT, ReviewDate DATE NOT NULL

Part 4. Normalization Workshop

The handout has no Part 3 (it goes from Part 2 to Part 4), so the numbering below follows the handout.

Task 4.1 Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

Assumptions: (A1) each project has exactly one supervisor; (A2) a student can work on many projects and a project has many students; (A3) a student has one role in a given project; (A4) Role, HoursWorked, StartDate and EndDate describe the student's participation in the project (if StartDate/EndDate were the dates of the whole project, they would move to the Project table).

1. Functional dependencies

FD	Dependency	Meaning
FD1	StudentID → StudentName, StudentMajor	A student has one name and one major.
FD2	ProjectID → ProjectTitle, ProjectType, SupervisorID	A project has one title, type and supervisor (A1).
FD3	SupervisorID → SupervisorName, SupervisorDept	A supervisor has one name and one department.
FD4	{StudentID, ProjectID} → Role, HoursWorked, StartDate, EndDate	Participation data depend on the student-project pair.
FD5 (implied)	ProjectID → SupervisorName, SupervisorDept	Follows from FD2 and FD3 by transitivity.

2. Problems: redundancy and anomalies

Redundancy. A student's name and major are repeated in every row for every project the student joins. The title, type and supervisor of a project are repeated for every student on the project. A supervisor's name and department are repeated for every student of every project they supervise.

Anomaly	Specific example
Update	Student S01 works on 3 projects, so her major appears in 3 rows. When she changes major from Physics to Computer Science, all 3 rows must be updated; if only 2 are changed, the database shows two different majors for the same student.
Insert	A new project P20 "Campus Energy Monitor" with supervisor F07 cannot be recorded until at least one student joins it, because StudentID is part of the primary key and cannot be NULL. Likewise a new supervisor who has no project yet cannot be stored.
Delete	Student S05 is the only member of project P12. If S05 leaves and the row is deleted, the title and type of P12 and its supervisor's name and department (if F03 supervises no other project) are lost as well.

3. First Normal Form (1NF)

A table is in 1NF when every cell holds a single atomic value, there are no repeating groups, and rows are identified by a key. As given, every attribute is single-valued and a key exists ({StudentID, ProjectID}), so **the table is in 1NF — there is no 1NF violation**. A violation would appear only if a cell stored a list, e.g. Role = "Developer, Tester". The fix would be one row per role (Role then joins the key) or a separate table for roles.

4. Second Normal Form (2NF)

Primary key: {StudentID, ProjectID}. Its closure contains all 13 attributes (FD1 + FD2 + FD3 + FD4), and neither attribute alone is enough: a student has many projects and a project has many students.

Partial dependencies (a non-key attribute depends on only part of the key):

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept

2NF decomposition (move each partial dependency into its own table):

Table	Definition (PK underlined, FK marked)
Student	<u>StudentID</u> , StudentName, StudentMajor
Project	<u>ProjectID</u> , ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept
StudentProject	<u>StudentID</u> FK → Student, <u>ProjectID</u> FK → Project, Role, HoursWorked, StartDate, EndDate

All three tables are in 2NF: in Student and Project the key has a single attribute, and in StudentProject every non-key attribute depends on the whole key.

5. Third Normal Form (3NF)

Transitive dependency in Project: ProjectID → SupervisorID → SupervisorName, SupervisorDept. The non-key attributes SupervisorName and SupervisorDept depend on another non-key attribute, SupervisorID. Move them to a Supervisor table.

Final 3NF decomposition:

Table	Definition (PK underlined, FK marked)
Student	<u>StudentID</u> INT, StudentName VARCHAR(100) NOT NULL, StudentMajor VARCHAR(60)
Supervisor	<u>SupervisorID</u> INT, SupervisorName VARCHAR(100) NOT NULL, SupervisorDept VARCHAR(60)
Project	<u>ProjectID</u> INT, ProjectTitle VARCHAR(150) NOT NULL, ProjectType VARCHAR(40), SupervisorID INT NOT NULL FK → Supervisor
StudentProject	<u>StudentID</u> INT FK → Student, <u>ProjectID</u> INT FK → Project, Role VARCHAR(40), HoursWorked DECIMAL(6,1) CHECK (HoursWorked ≥ 0), StartDate DATE, EndDate DATE, CHECK (EndDate IS NULL OR EndDate ≥ StartDate)

Check. In every table each determinant is the primary key, so the schema is in 3NF (and also in BCNF). Every FD (FD1–FD4) lies inside one table, so all dependencies are preserved. Each split was made on a determinant that became the key of the new table, so joining the tables back gives exactly the original data (lossless join). The anomalies disappear: a major is stored once; a project or supervisor can be inserted without students; deleting a participation row does not delete project or supervisor data.

Task 4.2 Advanced Normalization

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

Why this is tricky. The table has no SectionID, but the rules talk about course *sections*. A section must therefore be identified by a combination of the existing attributes. Since a course can be offered in several sections at different times, and one course is not taught twice in the same time slot, a section is identified by **(CourseID, TimeSlot)**.

2. Functional dependencies (listed first, because the key is derived from them)

FD	Dependency	Source
FD1	StudentID → StudentMajor	Each student has exactly one major.
FD2	CourseID → CourseName	Each course has a fixed name.
FD3	InstructorID → InstructorName	Each instructor has exactly one name.
FD4	Room → Building	Rooms are unique across campus. The stated rule {TimeSlot, Room} → Building follows from FD4, so FD4 is its minimal form.
FD5	{CourseID, TimeSlot} → InstructorID, Room	Each course section is taught by one instructor at one time in one room.
FD6	{TimeSlot, Room} → CourseID	Physical constraint (assumption): only one section can use a room in a given time slot.
FD7	{InstructorID, TimeSlot} → CourseID, Room	Physical constraint (assumption): an instructor cannot teach two sections at the same time.

1. Primary key

Closure of {StudentID, CourseID, TimeSlot}: start with {StudentID, CourseID, TimeSlot}; FD1 adds StudentMajor; FD2 adds CourseName; FD5 adds InstructorID and Room; FD3 adds InstructorName; FD4 adds Building. The closure contains all 9 attributes, so the set is a superkey.

Minimality: without StudentID we cannot know the student (no FD determines it); without TimeSlot we cannot tell which section of the course (the rules do not forbid a student from taking two sections of one course); without CourseID the time slot alone does not identify a section. Therefore **{StudentID, CourseID, TimeSlot}** is a candidate key.

Primary key: CourseSchedule(StudentID, CourseID, TimeSlot, ...) — “a student enrolled in a course section”.

Because of FD6 and FD7 there are two more candidate keys: **{StudentID, TimeSlot, Room}** (FD6 gives CourseID, then FD5 gives InstructorID) and **{StudentID, InstructorID, TimeSlot}** (FD7 gives CourseID and Room). The first one is chosen because it matches how a registration is described. Note: if the university also stated that a student takes only one section of a course, {StudentID, CourseID} would become the key.

3. BCNF check

A relation is in BCNF if, for every non-trivial FD $X \rightarrow Y$, X is a superkey. Every candidate key contains StudentID, so any determinant without StudentID is not a superkey:

FD	Is the left side a superkey?	Result
FD1 StudentID → StudentMajor	No (a student has many rows)	Violation (also a partial dependency, so not even 2NF)
FD2 CourseID → CourseName	No	Violation (partial dependency)
FD3 InstructorID → InstructorName	No	Violation
FD4 Room → Building	No	Violation

FD	Is the left side a superkey?	Result
FD5 {CourseID, TimeSlot} → InstructorID, Room	No (many students per section)	Violation (partial dependency)
FD6, FD7	No	Violation

Conclusion: CourseSchedule is not in BCNF (it is only in 1NF).

4. Decomposition to BCNF

Method: for a violating FD $X \rightarrow Y$, split R into $R_1 = X \cup Y$ and $R_2 = R - Y$. The common attributes X form a key of R_1 , so each split is lossless (Heath's theorem). Repeat until no violation remains.

Step	FD used	New table (BCNF)	Remaining table
1	FD1	Student(<u>StudentID</u> , StudentMajor)	R1(StudentID, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)
2	FD2	Course(<u>CourseID</u> , CourseName)	R2(StudentID, CourseID, InstructorID, InstructorName, TimeSlot, Room, Building)
3	FD3	Instructor(<u>InstructorID</u> , InstructorName)	R3(StudentID, CourseID, InstructorID, TimeSlot, Room, Building)
4	FD4	Room(<u>Room</u> , Building)	R4(StudentID, CourseID, InstructorID, TimeSlot, Room)
5	FD5	Section(<u>CourseID</u> , <u>TimeSlot</u> , InstructorID, Room)	Enrollment(<u>StudentID</u> , <u>CourseID</u> , <u>TimeSlot</u>)

Verification of the final tables:

- Student, Course, Instructor, Room: two attributes each, the only FD has the key on the left → BCNF.
- Section: the FDs FD5, FD6 and FD7 hold here. Its candidate keys are {CourseID, TimeSlot}, {TimeSlot, Room} and {InstructorID, TimeSlot}. The left side of every FD is a candidate key → BCNF.
- Enrollment: the key is all three attributes and there is no non-trivial FD → BCNF.

Final BCNF schema:

Table	Definition (PK underlined, FK marked)
Student	<u>StudentID</u> INT, StudentMajor VARCHAR(60) NOT NULL
Course	<u>CourseID</u> VARCHAR(10), CourseName VARCHAR(100) NOT NULL
Instructor	<u>InstructorID</u> INT, InstructorName VARCHAR(100) NOT NULL
Room	<u>Room</u> VARCHAR(10), Building VARCHAR(60) NOT NULL
Section	<u>CourseID</u> VARCHAR(10) FK → Course, <u>TimeSlot</u> VARCHAR(20), InstructorID INT NOT NULL FK → Instructor, Room VARCHAR(10) NOT NULL FK → Room; UNIQUE (TimeSlot, Room); UNIQUE (InstructorID, TimeSlot)
Enrollment	<u>StudentID</u> INT FK → Student, <u>CourseID</u> VARCHAR(10), <u>TimeSlot</u> VARCHAR(20); (CourseID, TimeSlot) FK → Section

5. Potential loss of information

- **No loss of data (lossless join).** Every split used a determinant that is a key of the new table, so $\text{Enrollment} \bowtie \text{Section} \bowtie \text{Student} \bowtie \text{Course} \bowtie \text{Instructor} \bowtie \text{Room}$ returns exactly the original rows, with no missing and no spurious tuples.
- **No loss of dependencies.** A BCNF decomposition *can* make an FD impossible to check inside one table (its attributes end up in different tables). This does not happen here: FD1-FD4 each lie in one table, and FD5-FD7 all lie in Section, where they are enforced by the primary key and the two UNIQUE constraints.

- **Information actually gained:** a course, instructor or room can now be stored before any student enrolls, and a section keeps its time and room when its last student drops it — the insert and delete anomalies of the original table are gone.
- **Costs and risks:** reconstructing the full schedule needs several joins. If the decomposition were done incorrectly, e.g. into (StudentID, TimeSlot, Room) and (CourseID, InstructorID, TimeSlot), the common attribute TimeSlot is not a key of either part and the join would create spurious rows (false enrollments). Finally, if the assumptions FD6/FD7 were false, the UNIQUE constraints on Section would have to be removed.

Part 5. Design Challenge

Task 5.1 Real-World Application: Student Clubs and Organizations

Assumptions: each club has exactly one faculty advisor; each event is organized by one club; a budget is allocated to a club per academic year; an expense is charged to one club budget and may be linked to an event; an event can have zero, one or several room reservations.

1. ER diagram

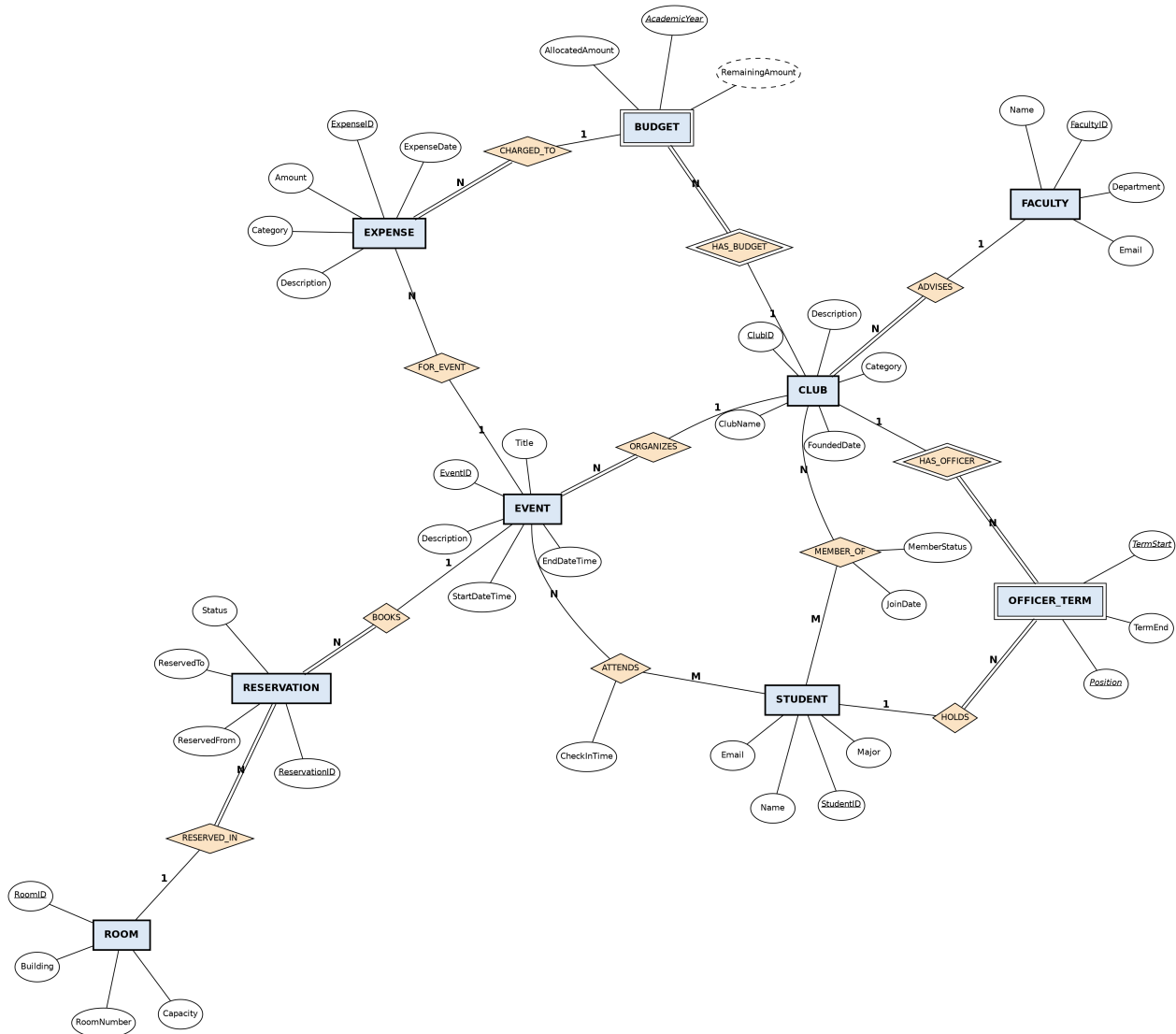


Figure 3. ER diagram of the student clubs system (Chen notation; legend in Part 2).

Relationship	Entities	Cardinality	Participation / attributes
MEMBER_OF	STUDENT - CLUB	M : N	partial; attributes JoinDate, MemberStatus
ATTENDS	STUDENT - EVENT	M : N	partial; attribute CheckInTime
ADVISES	FACULTY - CLUB	1 : N	CLUB total (every club has one advisor)
ORGANIZES	CLUB - EVENT	1 : N	EVENT total
HAS_officer (identifying)	CLUB - OFFICER_TERM	1 : N	OFFICER_TERM total; partial key (Position, TermStart)
HOLDS	STUDENT - OFFICER_TERM	1 : N	OFFICER_TERM total

Relationship	Entities	Cardinality	Participation / attributes
HAS_BUDGET (identifying)	CLUB – BUDGET	1 : N	BUDGET total; partial key AcademicYear
CHARGED_TO	EXPENSE – BUDGET	N : 1	EXPENSE total
FOR_EVENT	EXPENSE – EVENT	N : 1	partial (not every expense is for an event)
BOOKS	EVENT – RESERVATION	1 : N	RESERVATION total
RESERVED_IN	RESERVATION – ROOM	N : 1	RESERVATION total

2. Normalized relational schema

Mapping rules used: each strong entity → table; M:N relationship → table whose key combines both foreign keys (Membership, Attendance); 1:N relationship → foreign key on the N side; weak entity → table whose key includes the owner's key (OfficerTerm, Budget); the derived attribute RemainingAmount is not stored.

Table	Definition (PK underlined, FK marked)
Student	<u>StudentID</u> INT, Name VARCHAR(100) NOT NULL, Email VARCHAR(100) NOT NULL UNIQUE, Major VARCHAR(60)
Faculty	<u>FacultyID</u> INT, Name VARCHAR(100) NOT NULL, Email VARCHAR(100) NOT NULL UNIQUE, Department VARCHAR(60)
Club	<u>ClubID</u> INT, ClubName VARCHAR(100) NOT NULL UNIQUE, Description TEXT, Category VARCHAR(40), FoundedDate DATE, AdvisorID INT NOT NULL FK → Faculty(FacultyID)
Membership	<u>StudentID</u> INT FK → Student, <u>ClubID</u> INT FK → Club, JoinDate DATE NOT NULL, MemberStatus VARCHAR(20) NOT NULL CHECK (MemberStatus IN ('active', 'inactive', 'alumni'))
OfficerTerm	<u>ClubID</u> INT, <u>Position</u> VARCHAR(40), <u>TermStart</u> DATE, TermEnd DATE, StudentID INT NOT NULL; (StudentID, ClubID) FK → Membership(StudentID, ClubID); CHECK (TermEnd IS NULL OR TermEnd > TermStart)
Event	<u>EventID</u> INT, ClubID INT NOT NULL FK → Club, Title VARCHAR(120) NOT NULL, Description TEXT, StartDateTime TIMESTAMP NOT NULL, EndDateTime TIMESTAMP NOT NULL, CHECK (EndDateTime > StartDateTime)
Attendance	<u>StudentID</u> INT FK → Student, <u>EventID</u> INT FK → Event, CheckInTime TIMESTAMP
Room	<u>RoomID</u> INT, Building VARCHAR(60) NOT NULL, RoomNumber VARCHAR(10) NOT NULL, Capacity INT CHECK (Capacity > 0); UNIQUE (Building, RoomNumber)
Reservation	<u>ReservationID</u> INT, EventID INT NOT NULL FK → Event, RoomID INT NOT NULL FK → Room, ReservedFrom TIMESTAMP NOT NULL, ReservedTo TIMESTAMP NOT NULL, Status VARCHAR(20) NOT NULL; UNIQUE (RoomID, ReservedFrom); CHECK (ReservedTo > ReservedFrom)
Budget	<u>ClubID</u> INT FK → Club, <u>AcademicYear</u> CHAR(9) (e.g. '2026-2027'), AllocatedAmount DECIMAL(10,2) NOT NULL CHECK (AllocatedAmount ≥ 0)
Expense	<u>ExpenseID</u> INT, ClubID INT NOT NULL, AcademicYear CHAR(9) NOT NULL, EventID INT NOT NULL FK → Event, Amount DECIMAL(10,2) NOT NULL CHECK (Amount > 0), ExpenseDate DATE NOT NULL, Description VARCHAR(255), Category VARCHAR(40); (ClubID, AcademicYear) FK → Budget

Why the schema is normalized (BCNF, therefore also 3NF). Every table has atomic values (1NF). In the tables with composite keys (Membership, Attendance, OfficerTerm, Budget) the non-key attributes describe the whole key, so there are no partial dependencies (2NF). No non-key attribute determines another one: for example, Club stores only AdvisorID, while the advisor's name and email are kept in Faculty, so there is no transitive dependency (3NF). The only additional determinants are the UNIQUE columns (Student.Email, Faculty.Email, Club.ClubName, Room(Building, RoomNumber)), and each of them is itself a candidate key, so every determinant is a key (BCNF). The remaining budget is not stored, because storing AllocatedAmount –

SUM(Amount) would duplicate data and could become inconsistent.

The overlap of two reservations for the same room (e.g. 10:00–12:00 and 11:00–13:00) cannot be prevented by a UNIQUE constraint; it needs a CHECK trigger or an exclusion constraint. UNIQUE (RoomID, ReservedFrom) only blocks identical start times. Similarly, UNIQUE keys cannot stop two overlapping terms for the same officer position.

3. Design decision with several valid options

How to store officer positions (president, treasurer, secretary, ...).

Option	Description	Advantages	Disadvantages
A	Add a column Position to Membership.	Simple; no extra table or join.	A member can hold only one position; no history of past officers; Position is NULL for most members; nothing stops a club from having two presidents.
B (chosen)	Separate weak entity / table OfficerTerm(<u>ClubID</u> , <u>Position</u> , <u>TermStart</u> , TermEnd, StudentID).	Keeps the full history of terms; a student can hold several positions over time; the key prevents two records for the same position starting on the same date; the FK to Membership guarantees that only members become officers; no NULLs for ordinary members.	One extra table and join; overlapping terms need an extra check.

I chose **Option B** because universities usually need to know who was president in a previous year (for reports and signatures), and the design enforces the rule “officers must be members” directly with a foreign key.

Other decisions: (1) Budget is a weak entity per academic year instead of a single Budget column in Club, so each year’s allocation and spending can be compared. (2) Reservation is a separate entity instead of a RoomID column in Event, because an event may need no room (off-campus), several rooms or several time windows.

4. Example queries the database should support

- 1 “Show the current president and treasurer of every club together with the name and email of the club’s faculty advisor.” (*OfficerTerm with TermEnd IS NULL, joined with Student, Club and Faculty.*)
- 2 “For each club, show the budget allocated for 2026-2027, the total expenses recorded so far and the remaining balance, sorted from the smallest remaining balance.” (*Budget joined with the sum of Expense.Amount.*)
- 3 “List the students who attended at least three events of the Debate Club during Fall 2026 but are not registered members of that club.” (*Attendance and Event grouped by student, excluding students in Membership.*)